[Име на университета]
[Факултет / катедра]

КУРСОВ ПРОЕКТ

по дисциплина „Информационни системи“

Тема:

СИСТЕМА ЗА УПРАВЛЕНИЕ НА АВТОПАРК

Вариант 19

Изготвил: ……

Факултетен №: ……

Специалност: ……

Проверил: ……

2026 г.

Съдържание

1. Увод....3
1.1. Цел и обхват....3
1.2. Технологии....3
2. Анализ на изискванията....4
2.1. Функционални изисквания....4
2.2. Нефункционални изисквания....4
2.3. Роли и права....4
3. Проектиране....5
4. Моделиране (UML)....6
4.1. Use Case диаграма....6
4.2. ER диаграма....7
4.3. Клас диаграма....7
4.4. Последователност и дейности....10
5. База данни....11
6. Реализация....13
6.1. CRUD операции....13
6.2. Изчисляване на сроковете за обслужване....13
6.3. Проверка на километража....13
6.4. Гориво, курсове и номера....14
6.5. Справки и експорт....14
7. Потребителски интерфейс....15
8. Сигурност....20
9. Тестване....21
10. Инсталация и внедряване....21
11. Ръководство на потребителя....23
12. Заключение....23
Използвани източници....23

1. Увод

Разходите за гориво, обслужване, ремонти и застраховки са сред най-големите за фирма с много превозни средства, а пропуснат технически преглед или изтекла застраховка водят до глоби и престой. Когато информацията се води на хартия или в разпръснати таблици, е трудно да се отговори бързо на въпроси като „Кои коли имат преглед този месец?“ или „Колко ни струва всеки камион на километър?“.

Проектът (Вариант 19) реализира **Система за управление на автопарк** – уеб приложение, което дава цялостен поглед върху превозните средства: наличност и статус, предстояща поддръжка, история на ремонтите и разходите, шофьорите и ползването на автомобилите.

1.1. Цел и обхват

Системата поддържа регистър на МПС, план за периодична поддръжка с **автоматични напомняния по дата и по пробег**, пълна история на ремонтите, зарежданията и застраховките, шофьорите и курсовете, както и справки, които помагат при планирането. Таблицата показва как са покрити стъпките за изграждане на информационна система от заданието.

Таблица 1. Изпълнение на стъпките от заданието

Стъпка от заданието	Реализация в проекта	Раздел
Изисквания и проектиране	Функционални и нефункционални изисквания, трислойна архитектура	2, 3
Use Case, ER и клас диаграма	5 UML диаграми (+ последователност и дейности)	4
База данни	11 таблици в SQLite с външни ключове и ограничения	5
CRUD и бизнес логика	Всички обекти; напомняния, проверка на километража, суми, разход	6
Интерфейс	Уеб интерфейс на български (Bootstrap 5)	7
Тестване	154 автоматични теста (pytest), покритие 89 %	9
Документация и внедряване	Този документ, README.md, Docker и резервни копия	10, 11

1.2. Технологии

Приложението е написано на **Python 3** с уеб рамката **Flask 3.1**. Данните се съхраняват в **SQLite** чрез ORM библиотеката **SQLAlchemy 2**. Формулярите и защитата от CSRF са с **Flask-WTF**, удостоверяването – с **Flask-Login**, интерфейсът – с **Bootstrap 5.3** и **Chart.js 4**. Тестовете са с **pytest**, внедряването – с **gunicorn** и **Docker**, а диаграмите – с **PlantUML**.

2. Анализ на изискванията

Фирмата има служебни автомобили, бусове и камиони. Част от колите са поверени на конкретни служители, а останалите се отпускат на шофьор за курс. Всяко МПС има план за поддръжка – смяна на масло на 10 000 км или веднъж годишно, годишен технически преглед, сезонна смяна на гуми, спирачки, а камионите и проверка на тахографа. В работата участват механик, логистик и мениджър, а администраторът поддържа профилите.

2.1. Функционални изисквания

Таблица 2. Функционални изисквания и реализация

№	Изискване	Реализация
Ф1	Регистър на МПС: номер, марка/модел, година, тип, пробег, състояние	Екран „МПС“, статуси Активно / В ремонт / Спряно от движение
Ф2	План за поддръжка с интервали по км и дни	Видове дейности + индивидуален план за всяко МПС
Ф3	Напомняне при наближаване на срока	„Провери обслужване“, табло, график, календар (.ics)
Ф4	История на ремонтите: дата, описание, цена, км	Записи за обслужване с падащо меню на дейностите
Ф5	Разходи за гориво: дата, количество, цена	Зареждания с автоматична сума и среден разход
Ф6	Шофьори и кой кога е ползвал МПС	Шофьори, отговорен шофьор, отпускане и връщане от курс
Ф7	Справки: обслужвания, разходи, неактивни МПС	5 справки с филтри, графика, печат и CSV
Ф8	Застраховки и роли (по желание)	Застраховки „ГО“ и „Каско“; 4 роли с права

2.2. Нефункционални изисквания

Таблица 3. Нефункционални изисквания

Област	Как е постигнато
Целост на данните	Външни ключове, CHECK и уникални ограничения, проверка на последователността на километража
Използваемост	Български интерфейс, задължителни полета със *, автоматично попълване, цветна маркировка
Сигурност	Хеширани пароли, роли, CSRF, CSP, защита от SQL injection и XSS, заключване на профил
Надеждност	Docker с автоматичен рестарт, проверка на състоянието, ежедневен архив
Поддръжка	Разделение на слоеве, бизнес логика без зависимост от уеб слоя, тестове

2.3. Роли и права

Всеки влязъл потребител вижда МПС, историята и графика на обслужванията. Действията, които променят данни или показват финансова информация, зависят от ролята и се проверяват на сървъра.

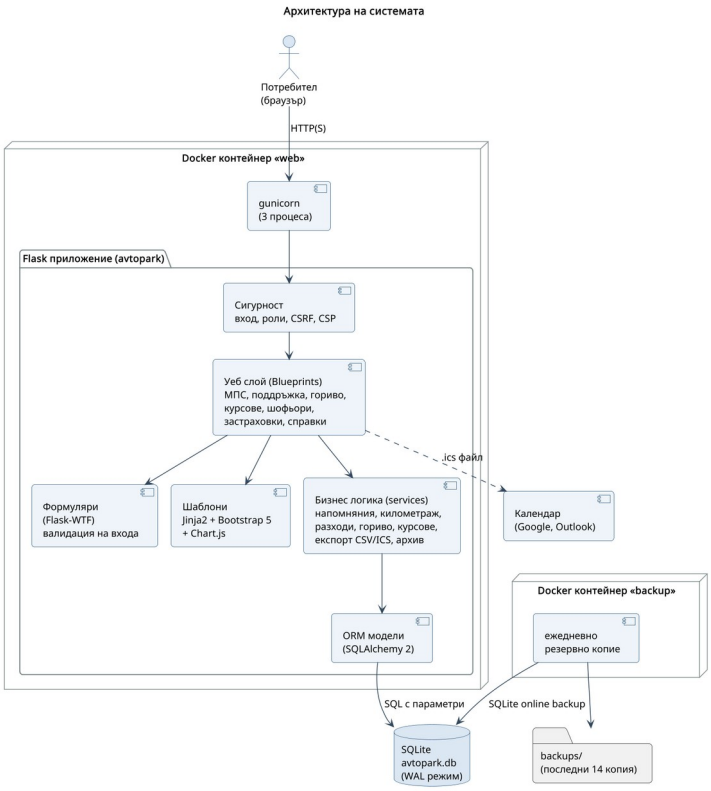
Таблица 4. Права на ролите

Действие	Механик	Логистик	Мениджър	Админ.
Преглед, „Провери обслужване“	да	да	да	да
Ремонти, план за поддръжка, статус на МПС	да	–	да	да
Гориво, курсове, шофьори	–	да	да	да

Действие	Механик	Логистик	Мениджър	Админ.
Регистър на МПС, застраховки, отчет на разходите, изтриване	–	–	да	да
Потребители и журнал на действията	–	–	–	да

3. Проектиране

Системата е уеб приложение с **трислойна архитектура**: уеб слой (Blueprints – приема заявката, проверява правата, валидира формуляра), **бизнес логика** (services – срокове, километраж, разходи, курсове, експорт; не зависи от HTTP и се тества лесно) и **слой за данни** (ORM модели с ограниченията на таблиците). Промените от една заявка се записват в една транзакция. Уеб вариантът позволява достъп от всеки компютър в офиса и сервиза без инсталация, а SQLite може да бъде заменена с PostgreSQL само чрез настройката `DATABASE_URL`.



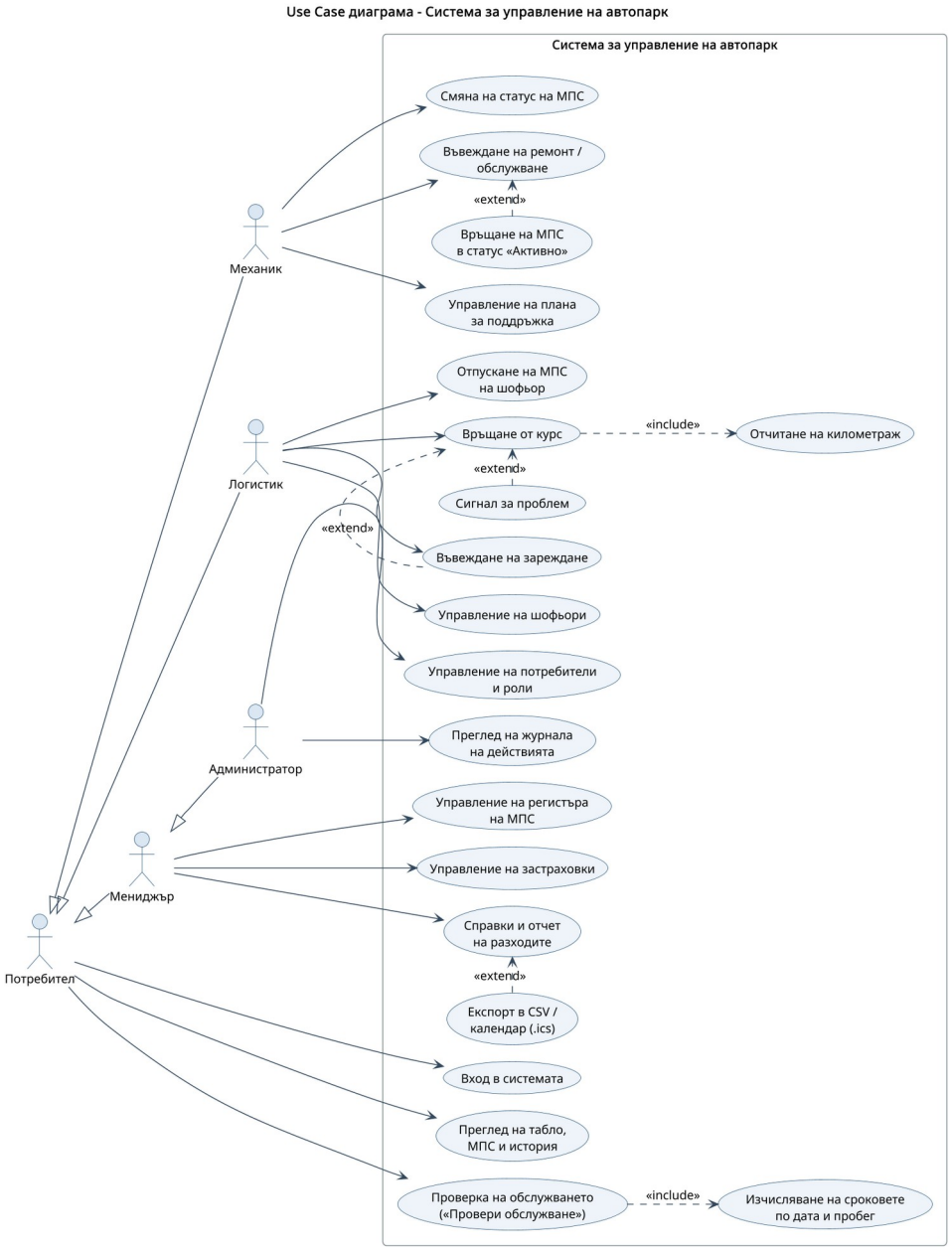
Фигура 1. Архитектура на системата

Кодът е организиран по слоеве: `models.py` (модели и ограничения), `forms.py` (валидация), `services/` (бизнес логика), `views/` (уеб слой по области), `templates/` (26 шаблона), `tests/` (154 теста) и `docs/` (диаграми и снимки). Приложението съдържа около 5 200 реда Python код, 1 800 реда шаблони и 1 200 реда тестове.

4. Моделиране (UML)

4.1. Use Case диаграма

Механикът, логистикът и мениджърът наследяват общите възможности на „Потребител“, а администраторът наследява мениджъра. „Include“ означава задължителна подфункция (връщането от курс винаги отчита километража), а „extend“ – допълнителна възможност (зареждане или сигнал за проблем при връщане).



Фигура 2. Use Case диаграма

Таблица 5. Основни случаи на употреба

Случай на употреба	Актьор	Основен сценарий
Проверка на обслужването	Всеки потребител	„Провери обслужване“ → сравнение на пробега и датата с интервалите → списък с просрочени и предстоящи дейности
Въвеждане на ремонт	Механик	Избор на МПС (км се попълва) → вид дейност от меню → цена и описание → по желание МПС става „Активно“

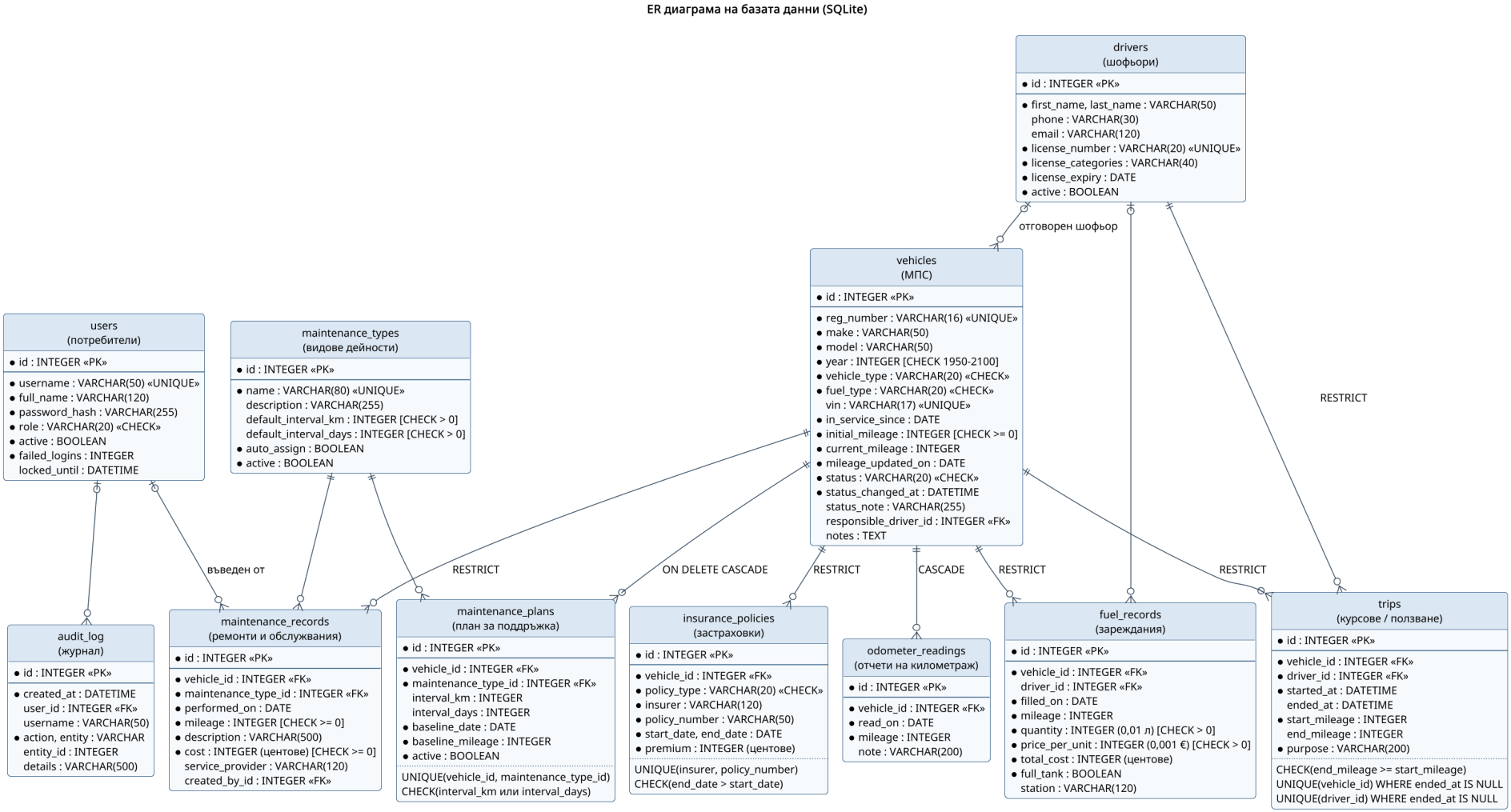
Случай на употреба	Актьор	Основен сценарий
Отпускане на МПС	Логистик	Свободно активно МПС и шофьор с валидна книжка → цел → курсът започва
Връщане от курс	Логистик	Краен км → по желание зареждане и проблем → пробегът се обновява; при проблем МПС отива „В ремонт“
Отчет на разходите	Мениджър	Период и МПС → гориво, обслужване, застраховки, €/км, графика → CSV
Управление на потребители	Администратор	Нов профил, роля, парола, деактивиране

4.2. ER диаграма

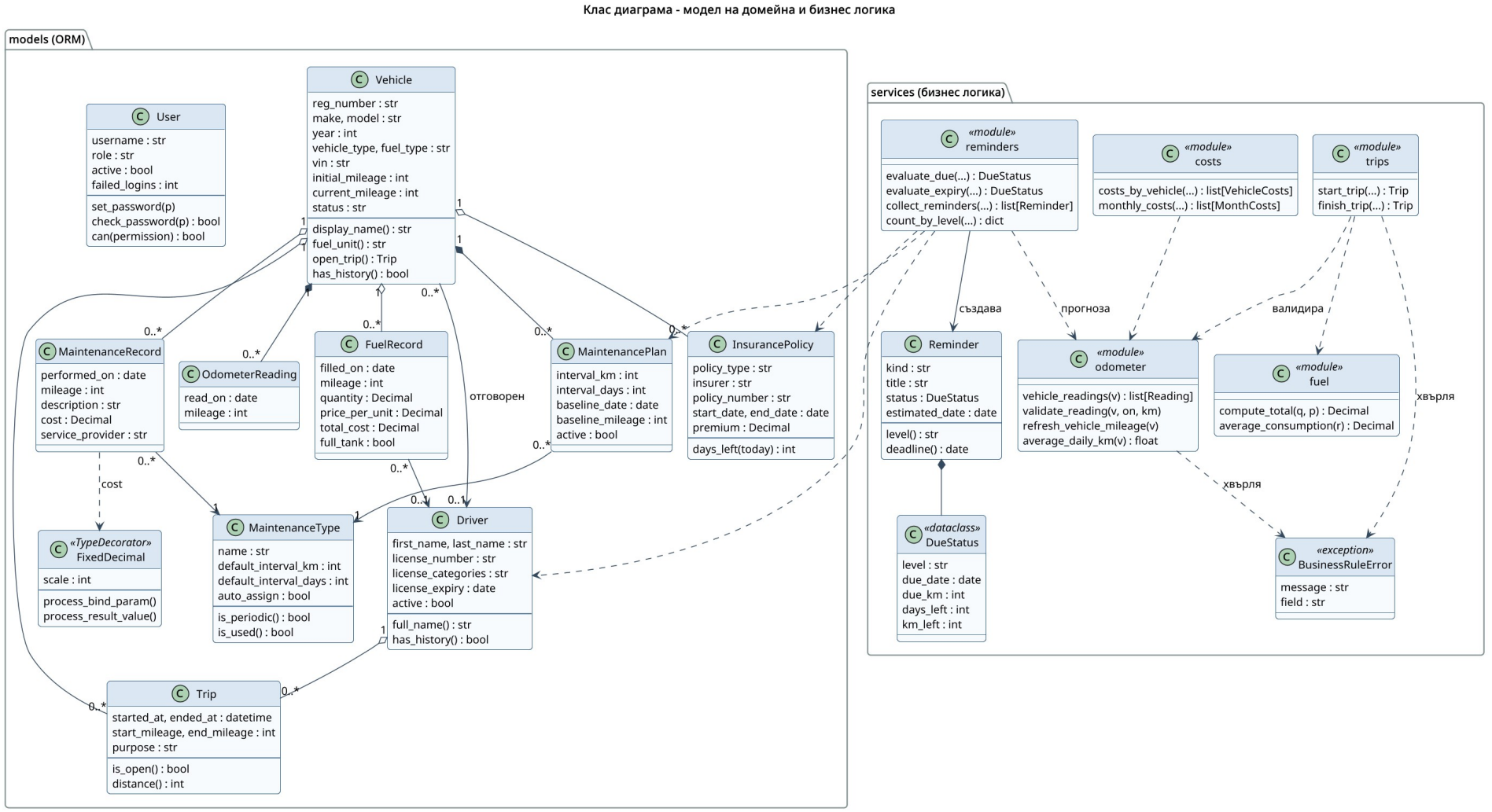
Централна е таблицата `vehicles`; към нея са свързани планът за поддръжка, ремонтите, зарежданията, курсовете, застраховките и отчетите на километража. До връзките е посочено поведението при изтриване: RESTRICT – забранено при свързани записи, CASCADE – изтриват се заедно. ER и клас диаграмата са на следващите две страници в хоризонтална ориентация.

4.3. Клас диаграма

Клас диаграмата показва ORM класовете с атрибути и методи и модулите на бизнес логиката. `Reminder` описва едно напомняне, а `DueStatus` – резултата от сравнението с интервала. При нарушено правило се хвърля `BusinessRuleError` със съобщение на български и името на полето.



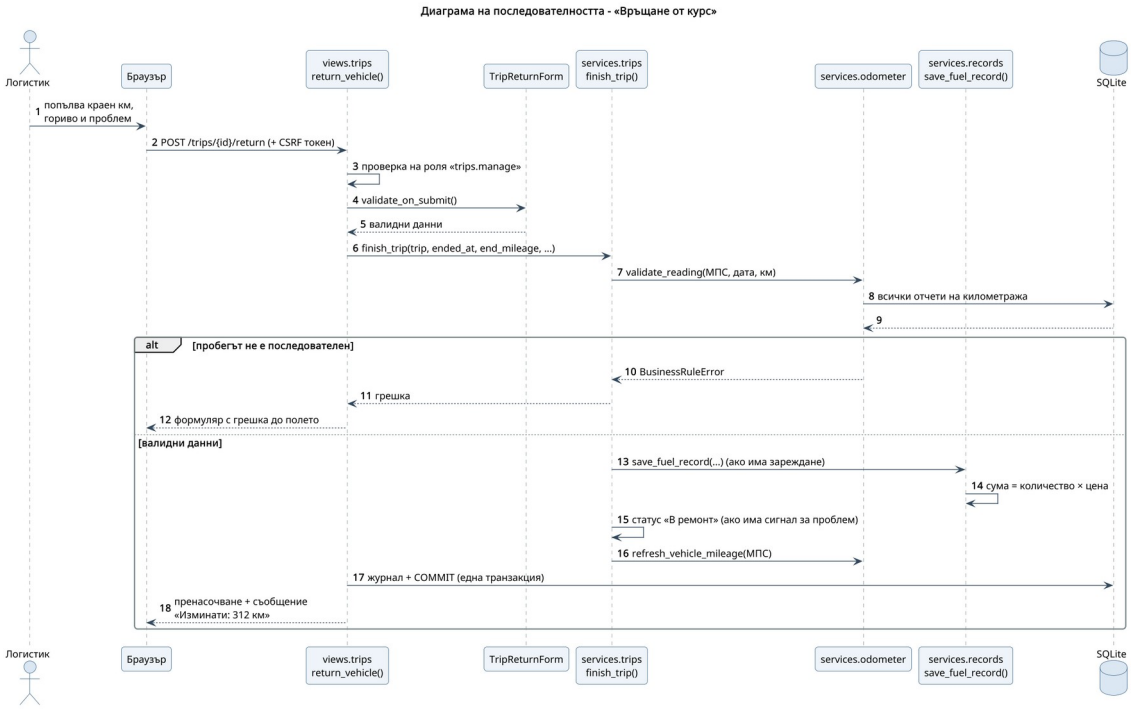
Фигура 3. ER диаграма на базата данни (11 таблици)



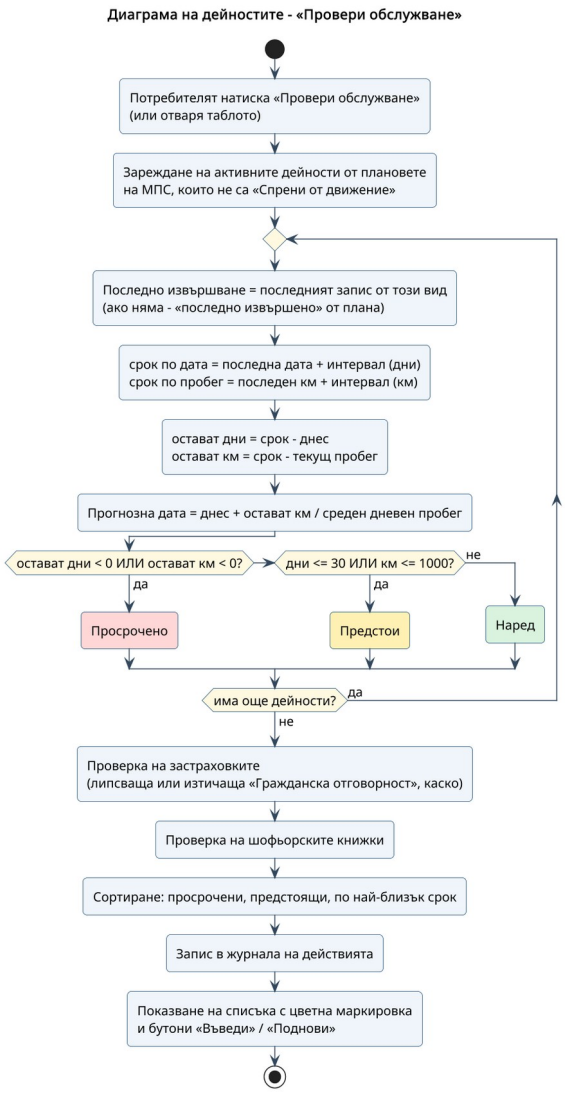
Фигура 4. Клас диаграма

4.4. Последователност и дейности

Диаграмата на последователността описва „Връщане от курс“: при непоследователен километраж потребителят получава грешка до полето и нищо не се записва; иначе в една транзакция се приключва курсът, записва се зареждането и се обновява пробегът. Диаграмата на дейностите описва алгоритъма на „Провери обслужване“ (раздел 6.2).



Фигура 5. Диаграма на последователността – „Връщане от курс“



Фигура 6. Диаграма на дейностите – „Провери обслужване“

5. База данни

Таблица 6. Таблици на базата данни

Таблица	Съдържание
vehicles	МПС: номер (UNIQUE), марка, модел, година, тип, гориво, VIN, пробег, статус, отговорен шофьор
drivers	Шофьори: име, телефон, имейл, № на книжка (UNIQUE), категории, валидност
maintenance_types	Видове дейности с интервали по подразбиране (км / дни)
maintenance_plans	План на МПС: дейност, интервал, последно извършване; UNIQUE (МПС, дейност)
maintenance_records	Извършени обслужвания и ремонти: дата, км, описание, цена, сервиз
fuel_records	Зареждания: количество, единична цена, сума, пълен резервоар
trips	Курсове: МПС, шофьор, начало и край, начален и краен километраж
insurance_policies	Застраховки „ГО“ и „Каско“: застраховател, номер, период, премия
odometer_readings	Ръчни отчети на километража
users, audit_log	Потребители с роли и хеширани пароли; журнал на действията

Целостта се осигурява на две нива – приложението валидира всеки вход и показва разбираемо съобщение, а базата отказва невалидни данни дори в обход на интерфейса:

- **външни ключове** (`PRAGMA foreign_keys = ON` при всяка връзка) – не може да има ремонт или курс за несъществуващо МПС;
- **ON DELETE RESTRICT** за историята – МПС с история не се изтрива, а се спира от движение; **CASCADE** само за плана;
- **CHECK** – допустими статуси и типове, неотрицателни суми и пробег, крайна дата след начална, краен км ≥ начален;
- **частичен уникален индекс** `WHERE ended_at IS NULL` – едно МПС и един шофьор са най-много в един незавършен курс.

SQLite няма точен десетичен тип, затова сумите се пазят като цели центове чрез собствен тип `FixedDecimal` и се връщат като точни `Decimal` стойности (3 × 0,10 € = 0,30 €). От 01.01.2026 г. валутата в България е еврото.

6. Реализация

6.1. CRUD операции

За МПС, ремонти, зареждания, курсове, шофьори, застраховки, планове, видове дейности и потребители са реализирани създаване, преглед (списъци с търсене, филтри и страниciране), редакция и изтриване. Изтриването минава през страница за потвърждение и се изпълнява само със заявка POST с CSRF токен; МПС и шофьори с история не се изтриват.

Таблица 7. CRUD операции по обекти

Обект	Създаване	Преглед	Редакция	Изтриване
МПС	формуляр	списък с търсене, карта	да + смяна на статус	само без история
Ремонт / обслужване	формуляр, бутон „Въведи“	история, филтри, сума	да	с потвърждение
Зареждане	формуляр, при връщане	списък, филтри, суми	да	с потвърждение
Курс	отпускане на МПС	история, филтри	да, връщане	с потвърждение
Шофьор / застраховка	формуляр	списък, карта	да	шофьор – само без история
Потребител	администратор	списък	роля, парола	деактивиране

6.2. Изчисляване на сроковете за обслужване

За всяка дейност от плана се определя **последното извършване** – последният запис от вида или „последно извършено“ от плана. Срокът по дата е последна дата + интервал в дни, а по пробег – последен километраж + интервал в км. Дейността е **просрочена**, ако е минат кой да е от двата срока, **предстои**, ако остават ≤ 30 дни или ≤ 1000 км, и е **наред** в останалите случаи. Точното достигане на срока се отчита като „Предстои“.

```
def evaluate_due(*, last_date, last_km, interval_days, interval_km,
                 today, current_km, days_ahead, km_ahead) -> DueStatus:
    due_date = last_date + timedelta(days=interval_days) if interval_days else None
    due_km = last_km + interval_km if interval_km else None
    days_left = (due_date - today).days if due_date else None
    km_left = due_km - current_km if due_km is not None else None
    if _passed(days_left) or _passed(km_left):            # < 0
        level = OVERDUE
    elif _within(days_left, days_ahead) or _within(km_left, km_ahead):
        level = DUE_SOON
    else:
        level = OK
    return DueStatus(level, due_date, due_km, days_left, km_left)
```

Системата изчислява и **прогнозна дата** по средния дневен пробег за последните 90 дни (днес + остават км ÷ км на ден), за да няма ръчни сметки. Проверката обхваща и застраховките (липсваща или изтекла „Гражданска отговорност“) и шофьорските книжки. МПС, спрени от движение, не се проверяват.

6.3. Проверка на километража

Всеки запис с километраж (ремонт, зареждане, начало и край на курс, ръчен отчет, начален пробег) е отчет на километража. Отчетите на едно МПС трябва да образуват ненамаляваща редица: датата не е в бъдещето, километражът не е по-малък от по-ранен

отчет и не е по-голям от по-късен, а ръст над 2000 км на ден се отхвърля като вероятна грешка (една нула в повече). Текущият пробег се преизчислява след всяка промяна, така че поправката на грешен запис възстановява правилната стойност.

6.4. Гориво, курсове и номера

Сумата на зареждането се изчислява автоматично и се закръгля до цент. Средният разход се смята по **метода „пълен резервоар“** – горивото между две пълни зареждания ÷ изминатите км × 100; за електромобили – в kWh. При **отпускане** се предлагат само активни свободни МПС и шофьори с валидна книжка; при **връщане** се въвеждат краен километраж, по желание зареждане и проблем (тогава МПС става „В ремонт“). Регистрационните номера, въведени с латински букви, се преобразуват в кирилица (`cb1234ab` → „СВ 1234 АВ“), за да няма дублирани МПС.

6.5. Справки и експорт

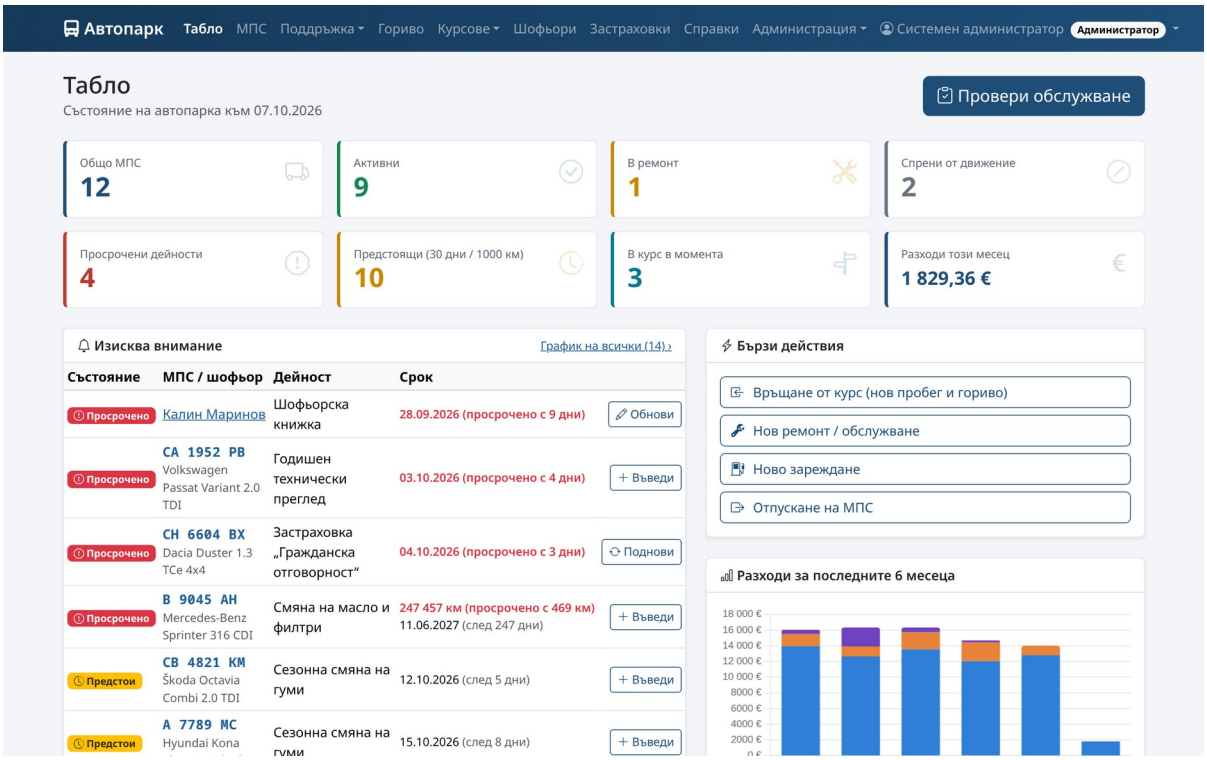
Таблица 8. Справки

Справка	Съдържание
График на обслужванията	Просрочени и предстоящи дейности, застраховки и книжки с оставащи дни/км; CSV и .ics
Отчет на разходите	Гориво, обслужване и застраховки по МПС за период, €/км, графика по месеци; CSV
Неактивни МПС	МПС в ремонт или спрени от движение – от кога и защо; активни МПС без движение
Разход на гориво	Количество, сума, средна цена и среден разход на 100 км по МПС
Ползване по шофьори	Кой шофьор кое МПС е ползвал, кога и колко км; CSV

CSV файловете са с разделител „;“ и UTF-8 BOM, за да се отварят в Excel с кирилица, а клетки, започващи с „=“, „+“, „-“ или „@“, се предпазват от изпълнение като формули. Файлът .ics (RFC 5545) се импортира в Google Calendar или Outlook – всяко напомняне става събитие с известие ден по-рано.

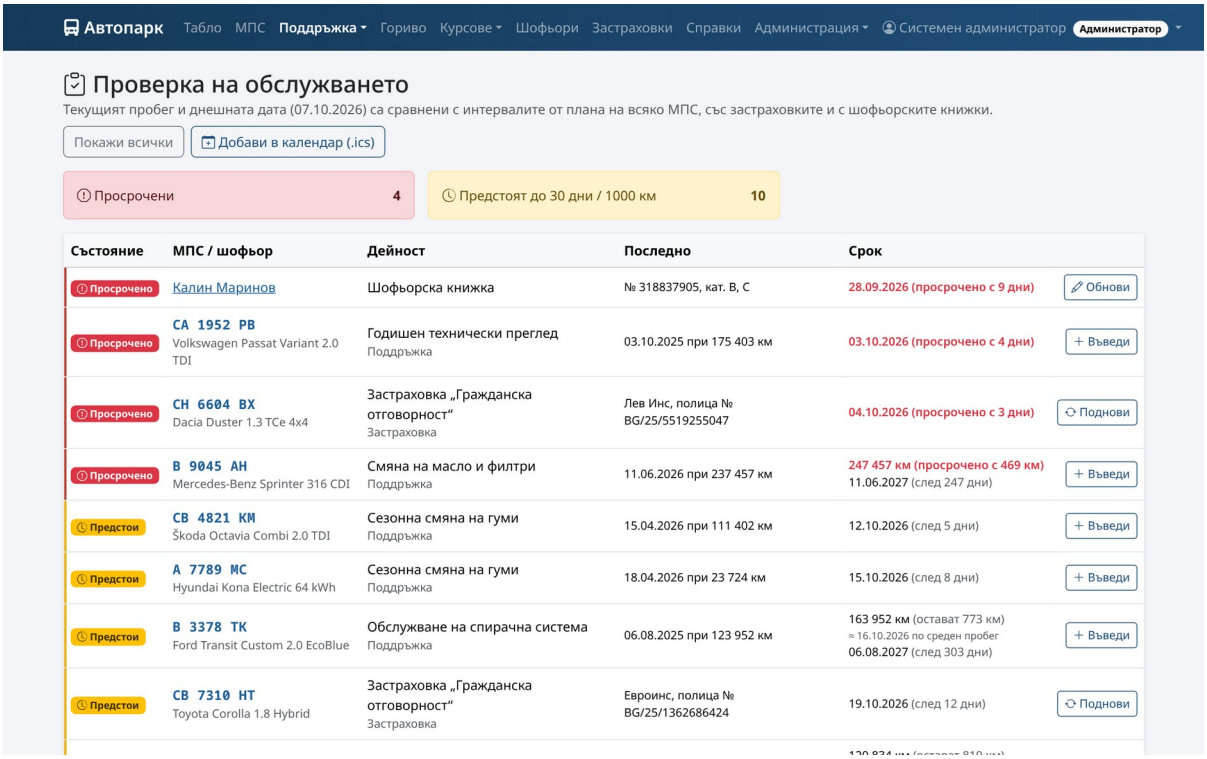
7. Потребителски интерфейс

Интерфейсът е на български, задължителните полета са отбелязани със звездичка, грешките се показват до полето, а датите и сумите са в познат формат (07.10.2026, 1 234,56 €). След вход се отваря **таблото** с обобщение на автопарка, списък „Изисква внимание“, бързи действия и графика на разходите.



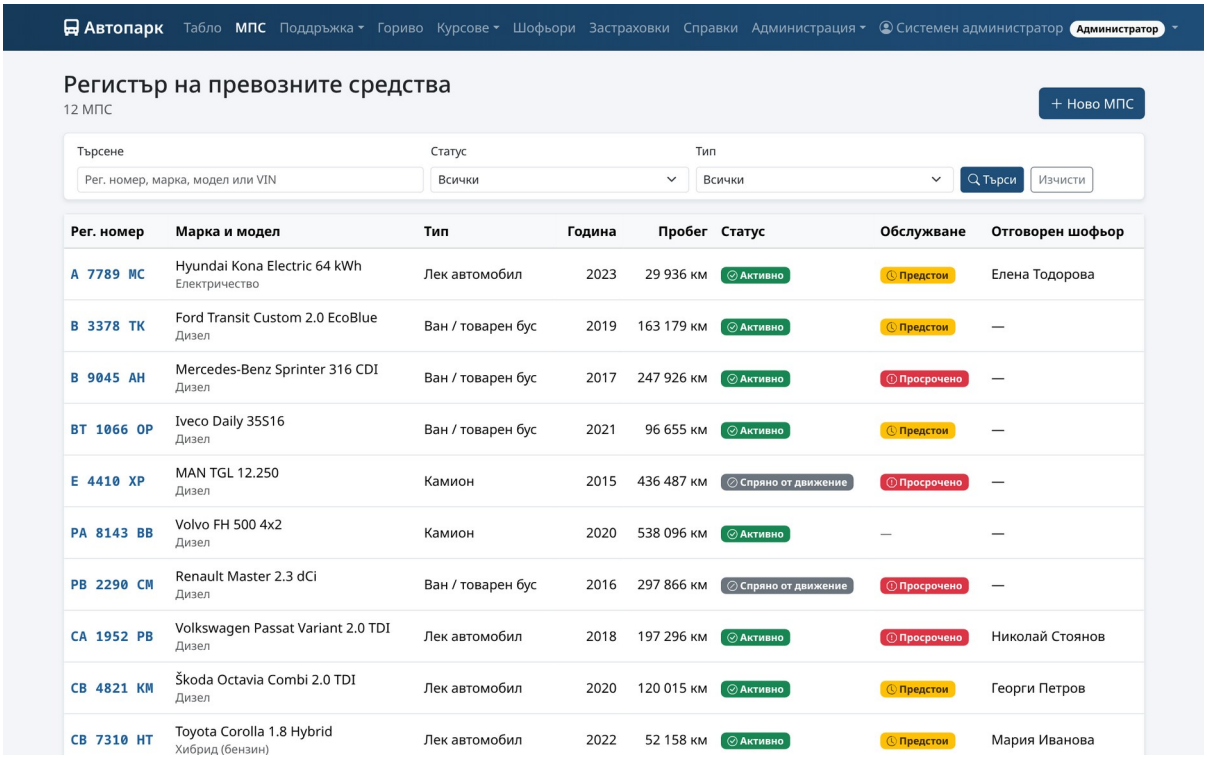
Фигура 7. Табло

Бутонът **„Провери обслужване“** показва просрочените дейности в червено и предстоящите в жълто; критерият, настъпил първи, е на първо място. Бутоните „Въведи“ и „Поднови“ отварят формуляра с вече избрани МПС и дейност.

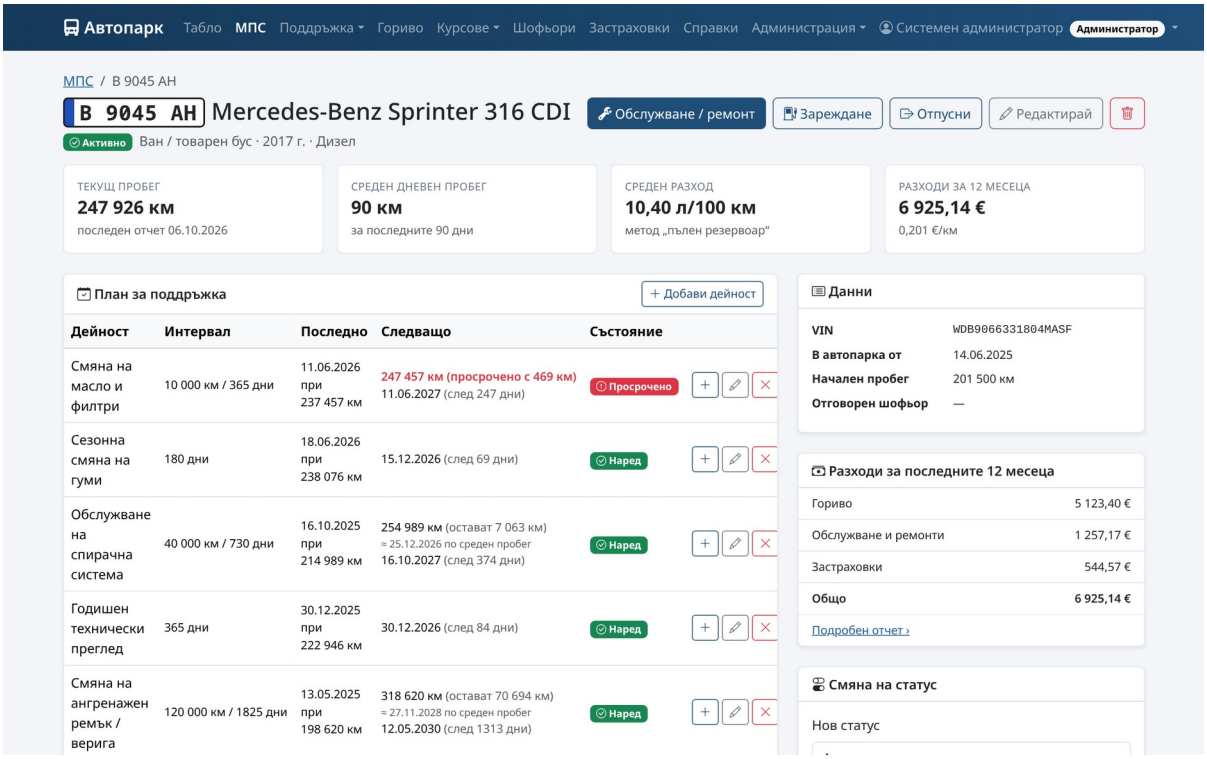


Фигура 8. Резултат от „Провери обслужване“

Картата на МПС събира текущия и средния дневен пробег, средния разход, разходите за 12 месеца, плана за поддръжка, застраховките и цялата история.

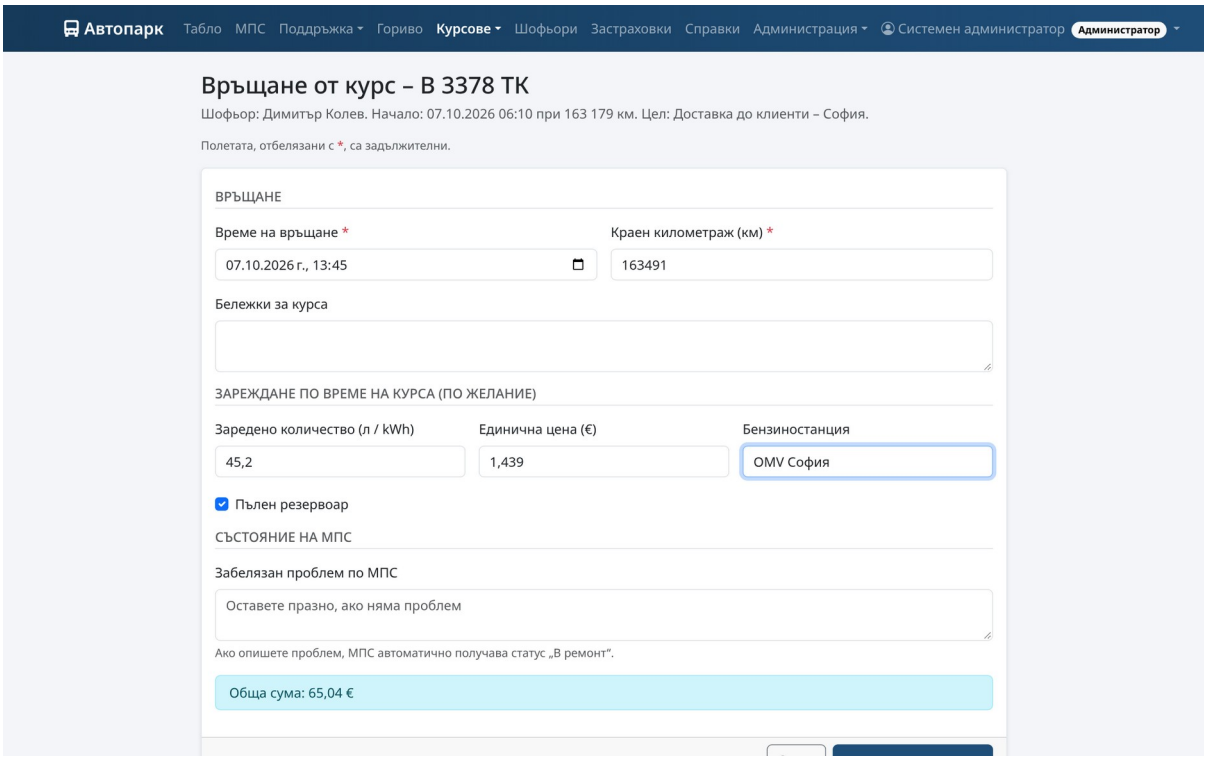


Фигура 9. Регистър на превозните средства

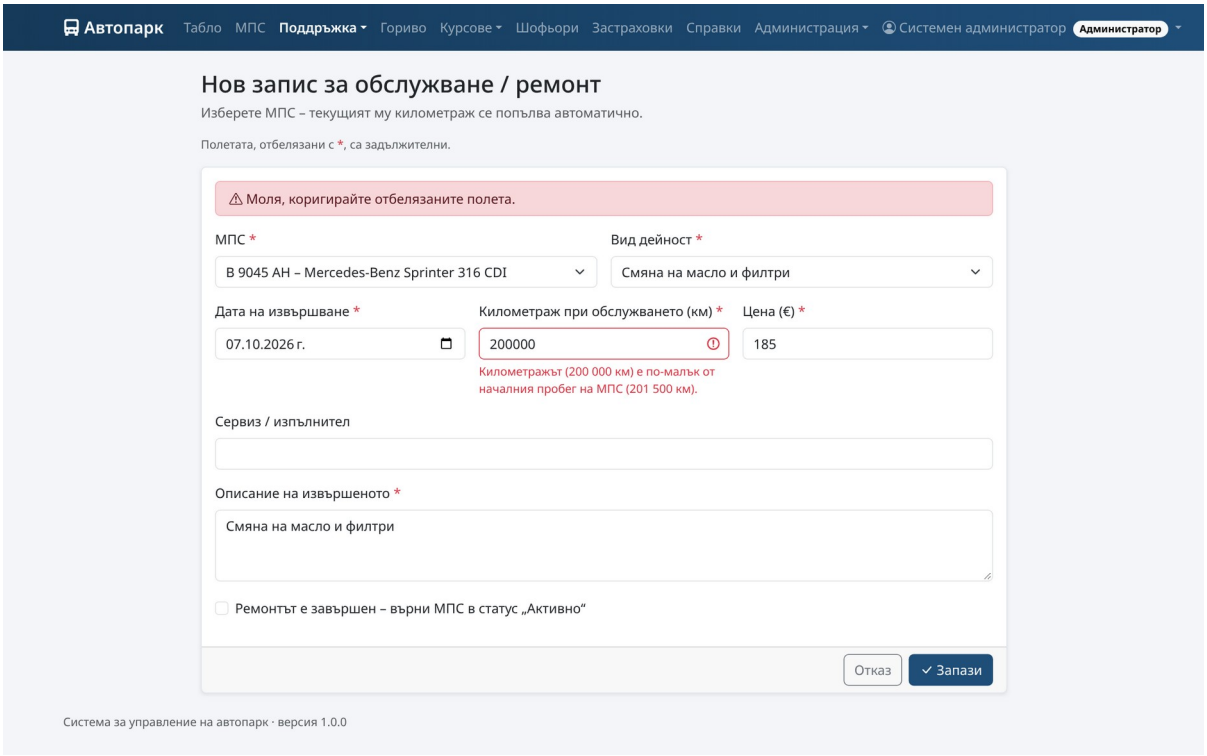


Фигура 10. Карта на МПС

При връщане от курс с един формуляр се въвеждат новият пробег, зареждането и забелязан проблем; сумата се изчислява докато потребителят пише. При непоследователен километраж грешката се показва до полето.

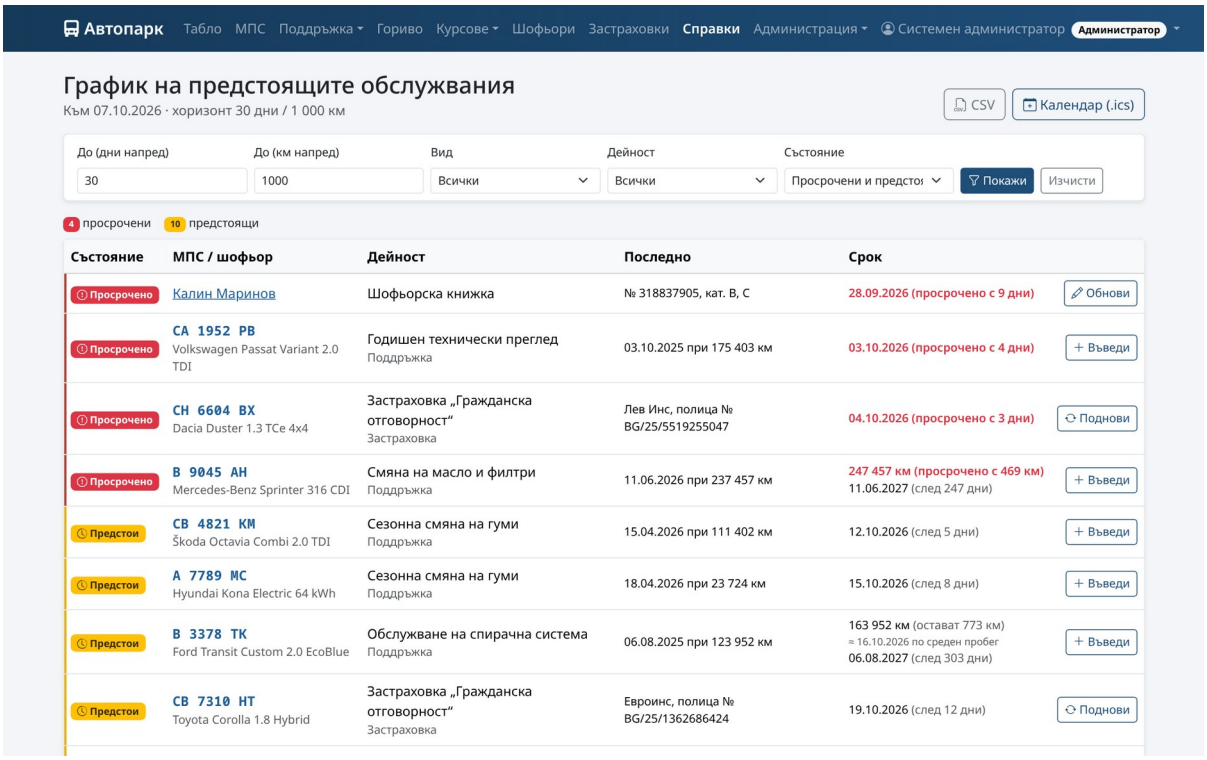


Фигура 11. Формуляр „Връщане от курс“

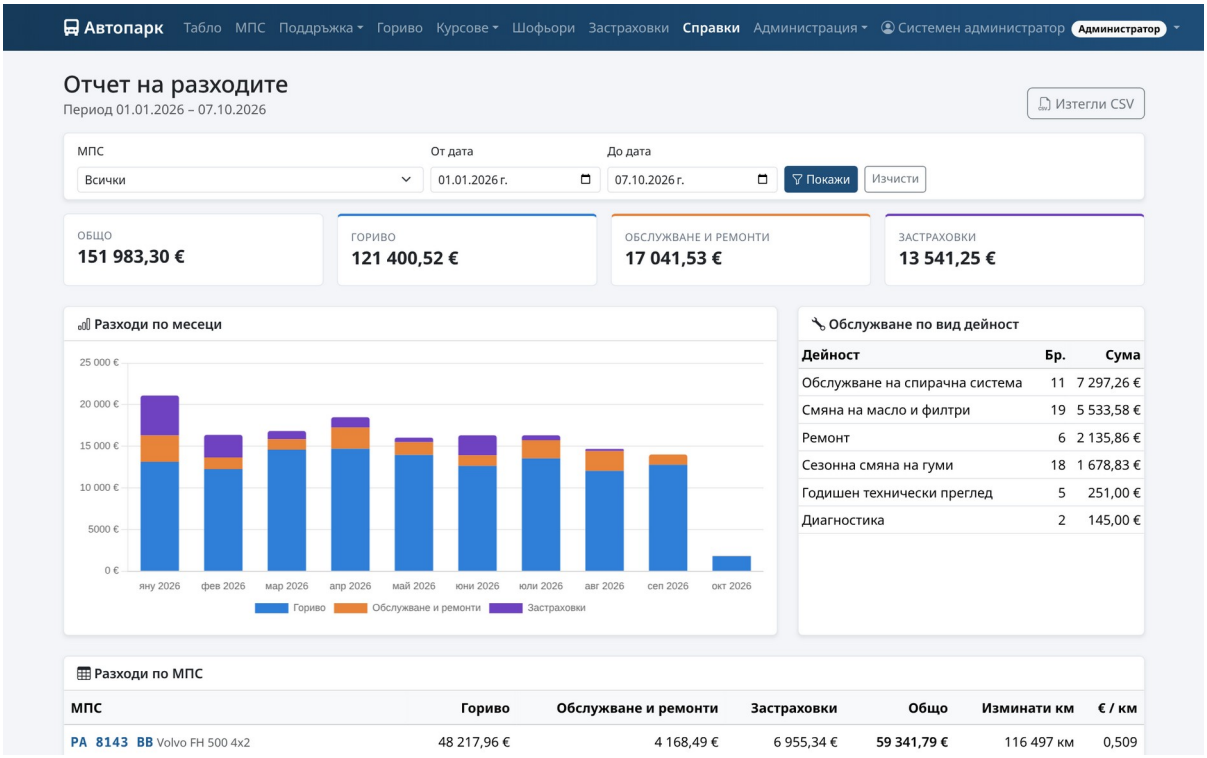


Фигура 12. Съобщение при непоследователен километраж

Справките имат филтри, печат и експорт. Графикът показва всички предстоящи дейности с оставащите дни и километри, а отчетът на разходите – суми по МПС, разход на километър и графика по месеци.



Фигура 13. График на предстоящите обслужвания



Фигура 14. Отчет на разходите с графика по месеци

8. Сигурност

Таблица 9. Заплахи и защитни мерки

Заплаха	Мярка в системата
Неоторизиран достъп	Всички страници освен входа и /health изискват вход; правата по роли се проверяват на сървъра (HTTP 403)
Отгатване на пароли	Мин. 8 символа с букви и цифри; заключване за 15 мин. след 5 грешни опита; еднакво съобщение за грешно име и парола
Изтичане на пароли	Пароли само като хеш (scrypt)
SQL injection	Всички заявки са чрез ORM с параметри
XSS	Автоматично екраниране в Jinja2; Content-Security-Policy без вградени скриптове
CSRF	CSRF токен във всеки POST формуляр; изход и изтриване само с POST
Сесии и заглавки	HttpOnly, SameSite, Secure при HTTPS; X-Frame-Options, nosniff; защита от open redirect
Лични данни	За шофьорите се пазят само нужните данни – без ЕГН и адрес
Загуба на данни	Ежедневни резервни копия; проверка на архива при възстановяване

Журналът `audit_log` пази входовете (вкл. неуспешните), създаването, промяната и изтриването на записи и всяка проверка на обслужването – с потребител, време и подробности.

9. Тестване

Тестовете са автоматични (`pytest`) и всеки получава собствена временна база. Има **модулни** тестове на бизнес логиката, **интеграционни** с базата данни и **функционални** през интерфейса (тестовият клиент на Flask).

Таблица 10. Тестови модули

Файл	Брой	Какво проверява
test_validators.py	30	номера, VIN, сила на паролата, български формат на числа и дати
test_reminders.py	21	срокове, гранични стойности, план и записи, застраховки, книжки
test_odometer_and_fuel.py	14	последователност на километража, среден пробег, разход на гориво
test_trips_costs_integrity.py	17	правила за курсове, разходи за период, FK, CHECK, RESTRICT
test_views.py	44	формуляри, грешни данни, потвърждение при изтриване, роли, експорт
test_management.py	11	шофьори, застраховки, дейности, планове, потребители, команди
test_security.py	17	вход, заключване, CSRF, XSS, SQL injection, заглавки, архив

Таблица 11. Избрани тестови сценарии – нормални и гранични случаи

Сценарий	Входни данни	Очакван резултат
Граница по пробег	сега 58 999 / 59 000 / 60 000 / 60 001 км (срок 60 000)	Наред / Предстои / Предстои / Просрочено
Граница по дата	последно преди 334 / 335 / 365 / 366 дни	Наред / Предстои / Предстои / Просрочено
Което настъпи първо	пробегът е надвишен, датата е след 247 дни	Просрочено по пробег
Намаляващ километраж	отчет 15 000 км след отчет 20 000 км	грешка, нищо не се записва
Една нула в повече	+2 001 км за един ден	грешка за нереалистичен пробег
Несъществуващо МПС	vehicle_id = 9999	отказ във формуляра и в базата (FK)
МПС в два курса	второ отпускане на същото МПС	отказ; базата също отказва
Точност и формат	3 × 0,10 €; цена „1 250,40“	0,30 €; записано 1250,40 €
Сигурност	XSS в бележка; POST без CSRF; 5 грешни пароли	екранирано; HTTP 400; заключен профил
Права	механик → нов запис за гориво	HTTP 403

Всички **154 теста преминават успешно** за около една минута. Покритието на кода (pytest-cov) е **89 %**, а на бизнес логиката (напомняния, километраж, разходи) – над 95 %. Приложението е проверено и ръчно в Google Chrome с демонстрационните данни, без грешки в конзолата.

10. Инсталация и внедряване

Локално (Python 3.12+):

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
flask --app avtopark seed-demo --yes   # демонстрационен автопарк
flask --app avtopark run               # http://127.0.0.1:5000
```

Демо данните съдържат 12 МПС, 12 шофьори и около 16 месеца история; потребители `admin`, `manager`, `mechanic`, `logistic` с парола `Demo1234`. За реална работа базата се създава празна с `flask --app avtopark init-db`.

На сървър: `cp .env.example .env` (задават се SECRET_KEY и ADMIN_PASSWORD) и `docker compose up -d --build`. Услугата **web** работи с gunicorn (3 процеса), автоматичен рестарт и проверка `/health`; базата е в отделен том и не се губи при обновяване. Услугата **backup** прави резервно копие всеки ден и пази последните 14. Ръчно: `flask --app avtopark backup` и `restore ФАЙЛ` (текущата база се запазва преди възстановяването). Командата `check-reminders` може да се пуска от cron и връща код 1 при просрочени дейности. За достъп отвън се използва reverse proxy с HTTPS.

11. Ръководство на потребителя

Таблица 12. Основни действия по роли

Роля	Как се работи
Всички	Табло → „Провери обслужване“ показва какво е спешно; МПС → търсене и карта на МПС с цялата история
Механик	Поддръжка → „Нов запис“ (или „Въведи“ до напомнянето): избор на МПС (км се попълва), вид дейност, цена, описание; отметка „Ремонтът е завършен“ връща МПС в „Активно“. Статус и план – от картата на МПС
Логистик	Курсове → „Отпускане на МПС“ (само свободни активни МПС) и „Връщане от курс“ (краен км, зареждане, проблем); Гориво → „Ново зареждане“ („Пълен резервоар“ за точен разход); Шофьори
Мениджър	Справки → разходи за период (CSV), график и календар (.ics), неактивни МПС; МПС → „Ново МПС“ (номерът може и на латиница); Застраховки → „Поднови“
Администратор	Администрация → Потребители (роля, парола, деактивиране) и Журнал на действията

Всички списъци и справки могат да се отпечатат с Ctrl+P – менюто и бутоните се скриват при печат.

12. Заключение

Създадена е работеща система за управление на автопарк, която покрива всички функционални изисквания на Вариант 19 – регистър на МПС, план за поддръжка с напомняния по дата и по пробег, история на ремонтите и разходите, шофьори и справки – както и препоръчителните разширения: роли, застраховки, графика на разходите и интеграция с календар. Качеството е потвърдено от 154 автоматични теста, а внедряването е автоматизирано с Docker и ежедневни резервни копия. Възможно развитие: напомняния по имейл и SMS, мобилен изглед за шофьорите, импорт от карти за гориво и GPS, миграция към PostgreSQL.

Използвани източници

- Flask 3.1 и SQLAlchemy 2.1 Documentation – https://flask.palletsprojects.com/, https://docs.sqlalchemy.org/
- SQLite Documentation (Foreign Keys, Partial Indexes, Online Backup API) – https://www.sqlite.org/docs.html
- OWASP Top 10 и OWASP Cheat Sheet Series – https://owasp.org/
- RFC 5545 – iCalendar; PlantUML Language Reference Guide – https://plantuml.com/
- Наредба № Н-32 за периодичните прегледи за проверка на техническата изправност на ППС